

Projeto OFB - Open Finance Brasil 2025

Projeto de autoria própria com a finalidade de estudo dos fluxos principais do Open Finance Brasil e revisão de todo conhecimento técnico de linguagens e processos.

Este documento apresenta o Escopo, Módulos e Funcionalidades e detalha a Estrutura do Projeto construído

O projeto fonte está disponível em [GitHub-ProjetoOFB](#)

Autor: fras200627

Data: jan-dez 2025

Apresentação

Este projeto foi desenvolvido para explorar as tecnologias usadas na construção de APIs, orquestradores, mensageria, processos em lote e bibliotecas.

Foi utilizada a linguagem Java e os Frameworks Springboot, SpringSecurity, SpringSessionJdbc, SpringCloud, Springbatch e SpringApacheCamel, bem como os recursos fornecidos pela OpenAPI 3.0.3, bancos de dados Oracle e MySQL e integração de dados com RabbitMQ.

Os estudos também implementam ambientes de execução baseados em Docker e Kubernetes, ambos rodando em Linux Ubuntu versões LTS 22.0.4 e 24.0.4 dentro componente WSL (Windows Subsystem for Linux) do Windows.

As funcionalidades foram baseadas nos fluxos de processamento existentes na [Página Principal do Open Finance Brasil](#).

Escopo do Projeto

- Apenas 4 fluxos do OFB foram utilizados, especificamente os 4 principais:
 - [Resources](#)
 - [Consents](#)
 - [Customers](#)
 - [Accounts](#)
- Os recursos acima foram construídos com todos os serviços existentes no [Site Oficial da Documentação para o Desenvolvimento OFB](#).
- A solução apresentada no "Projeto OFB" possui componentes para:
 - **Cloud**

- **Security**
- **Participant Directory simulation**
- **Orchestration**
- **Backend APIs**
- **Auditing**
- **Listeners**
- **Batches**

Notas: "O Diretório de participantes é uma plataforma centralizada que registra as organizações participantes no ambiente real do Open Finance Brasil."*

- **Sobre os recursos "Security" utilizados**

Os recursos Financial-grade API (FAPI) da OpenID Foundation, OAuth 2.0 e OpenID Connect (OIDC) existentes no ambiente real do OFB foram simulados pelos recursos disponibilizados no SpringSecurity junto com o complemento **SpringSessionJDBC**.

- **Sobre as funções do "Participant Directory simulation" utilizados**

As definições do "Diretório de Participantes" existentes no OFB real foram simulados no desenvolvido desse Projeto pelos recursos SpringSecurity junto com o complemento **SpringSessionJDBC**.

No complemento "**SpringSessionJDC**" é fornecido um modelo de dados completo para controle de Clients (participantes) com cadastro de Participants, Authorization Grant Types, Roles e Scopes.

Estrutura do Projeto - Suporte para as Funcionalidades

- As funcionalidades dos componentes são suportadas por **bancos de dados Oracle e MySQL, mensageria via RabbitMQ e processamentos em lotes**.
 - O banco de dados Oracle é o repositório de **informações sobre os domínios de controle do OFB e o repositório de solicitações de consentimento, bem como o controle de recursos/permissões dos consentimentos**.
 - O banco de dados MySQL é o repositório de **informações sobre controle de segurança/tokens/autenticação de acesso para OFB e também para informações de rastreamento de auditoria**.
 - A mensageria conecta **as atividades de todos os componentes com os processos sendo registrados em logs de auditoria**.
 - Os processos em lote **simulam a "Aprovação de Consentimentos" e controlam a "Expiração do Consentimentos"**.

Estrutura do Projeto - Persistência, Desempenho e Acesso a Dados

- Persistência
 - O repositório de dados transacionais está no Oracle e os mecanismos de persistência são implementados pelo Spring Data JPA.
- Desempenho e Acesso aos Dados
 - O acesso aos dados transacionais utiliza a solução de objetos **"Materialized Views" da Oracle** e essas **"visualizações"** são consumidas pela implementação Spring Data JPA nos códigos das APIs.
 - A consistência dos status e permissões dos "CONSENTS" e dos "RESOURCES" utilizam as soluções de objetos "Triggers" e "Procedures" da Oracle.

Sobre as "Materialized Views" da Oracle

As "Materialized Views" da Oracle são objetos classificados como Data Warehouse (DW) e são objetos "views" reais criados e mantidos nos TableSpace do Oracle.

Essas "Views" consolidam os dados transacionais e fornecem resultados já agrupados e classificados, permitindo uma codificação de APIs extremamente reduzida e simples.

As "Materialized Views" também fornecem alto desempenho para acesso a dados pois são objetos executados diretamente "na memória" do Host/Instance Oracle.

As "Materialized Views" são mantidas diretamente pelos "Oracle Engines" em processos executados em real-time, o que garante o estado atual dos dados transacionais.

Estrutura do Projeto - Modulos

1. **Gerenciamento**
 - Discovery Services
 - API Gateway
2. **Serviços de Segurança de Acessos**
 - Authentication Server
 - Authorization Server
3. **Serviços Específicos do OFB**
 - Consents
 - Customers
 - Accounts
 - Resources
4. **Serviços complementares para o OFB**
 - Participants Clients (simulation of Participants Directory)
 - Audit Tracking

- Audit Authorization Consents
- Audit Cancellation Consents
- Audit Extends Consents
- Audit Revoked Consents

5. Serviços de Mensageria

- Audit Services
- Consents Authorization Services
- Consents Cancellation Services

6. Processos Batch

- Consents Approval Control
- Consents Expiration Control

Estrutura do Projeto - Databases

- Repositórios de dados
 - **Oracle, objetos do esquema "ofb"**
 - . Tabelas de dados para domínios OFB (controles principais)
 - . Tabelas de dados para consentimentos, recursos, clientes e contas
 - . Tabelas de dados exigidas pelos controles dos componentes SpringBatch
 - **MySQL, objetos do esquema "ofb"**
 - . Tabelas de dados para os logs de auditoria
 - **MySQL, objetos do esquema "user"**
 - . Tabelas de dados exigidas pelos controles dos componentes SpringSecurity e SpringCloud
- Mecanismos de dados
 - **Objetos Oracle Materialized Views**
 - . Visualizações de objetos para implementar o acesso a dados transacionais
 - **Objetos Oracle Triggers**
 - . Gatilhos para implementar controles de dados de atualização automática
 - **Objetos Oracle Procedures**
 - . Procedimentos para implementar a manutenção automática do controle de dados.

Estrutura do Projeto - Componentes

Notas:

- **lib** >> notação usada nos nomes dos componentes de bibliotecas
- **atom** >> notação usada nos nomes dos componentes de APIs de backend
- **camel-orch** >> notação usada nos nomes dos componentes de APIs de orquestração
- **listener** >> notação usada nos nomes dos componentes dos serviços de mensagens
- **sbatch** >> notação usada nos nomes dos componentes em lote

1. 01 - Libs Components

- sbboot-**lib**-ofb-autoconfigure-amqp
 - sbboot-**lib**-ofb-autoconfigure-handlers
 - sbboot-**lib**-ofb-autoconfigure-interceptors
 - sbboot-**lib**-ofb-autoconfigure-security
 - sbboot-**lib**-ofb-common-tools
2. ☒ 02 - **Backend APIs**
- sbboot-**atom**-ofb-consents
 - sbboot-**atom**-ofb-resources
 - sbboot-**atom**-ofb-customers
 - sbboot-**atom**-ofb-accounts
 - sbboot-**atom**-ofb-participants-domain
 - sbboot-**atom**-ofb-participants-business
 - sbboot-**atom**-ofb-audit-tracking
 - sbboot-**atom**-ofb-audit-authorization-consents
 - sbboot-**atom**-ofb-audit-cancellation-consents
 - sbboot-**atom**-ofb-audit-extends-consents
 - sbboot-**atom**-ofb-audit-revoked-consents
3. ☒ 03 - **Managment and Security Components**
- sbboot-**atom**-ofb-discovery-services
 - sbboot-**atom**-ofb-api-gateway
 - sbboot-**atom**-ofb-authentication-server
 - sbboot-**atom**-ofb-authorization-server
4. ☒ 04 - **Orchestration Components**
- sbboot-**camel-orch**-ofb-consents
 - sbboot-**camel-orch**-ofb-resources
 - sbboot-**camel-orch**-ofb-customers
 - sbboot-**camel-orch**-ofb-accounts
5. ☒ 05 - **Listeners Services**
- sbboot-**listener**-ofb-audit-services
 - sbboot-**listener**-ofb-consents-authorization-services
 - sbboot-**listener**-ofb-consents-cancellation-services
6. ☒ 06 - **Batches Services**
- **sbbatch**-ofb-consents-approval-control
 - **sbbatch**-ofb-consents-expiration-control

Ambientes de Execução do Projeto

Dev (local)

Nota: **APIs, Listeners and Batches started in IntelliJ**

- Windows 11
- Oracle XE 21 for Windows 11 in local instalation
- MySQL 8.0 for Windows 11 in local installation
- RabbitMQ for Windows 11 in local installation

Homologação (docker)

- Docker running into WSL
- Windows Sub-System Linux (Ubuntu 22.04 LTS)
- Oracle XE 21 image running in Docker
- MySQL 8.0 image running in Docker
- RabbitMQ 3.XX image running in Docker
- OFB APIs, Listeners and Batches images running in Docker

Produção (microk8s)

- Kubernetes (plugins MicroK8S) running into WSL
- Windows Sub-System Linux (Ubuntu 24.04 LTS)
- Oracle XE 21 image running in Kubernetes
- MySQL 8.0 image running in Kubernetes
- RabbitMQ 3.XX image running in Kubernetes
- OFB APIs, Listeners and Batches images running in PODs

Apps, Frameworks e Plugins Usados

Ambientes

- Windows 11 (hardware core: Intel Core I7 8th, 32Gb ram and SSD 512Gb)
- WSL - Windows SubSystem Linux - Ubuntu Server 22.04 LTS (run Docker)
- WSL - Windows SubSystem Linux - Ubuntu Server 24.04 LTS (run Kubernetes)
- Docker
- Kubernetes (microK8S pack)

Databases Usados

- Oracle XE core version 21
- MySQL CE core version 8.0

Serviços de Integração Usados

- RabbitMQ 3.x

Ferramentas Usadas no Desenvolvimento

- IntelliJ
- OpenAPI (3.0.3)
- Swagger and Swagger Editor
- Portainer
- PL/SQL Development
- MySQL Workbench
- RabbitMQ Service Tools

Serviços usados para Gerenciamento do Projeto

- Git Hub
- Docker Hub
- Maven
- OpenAPI

Frameworks and Plugins Usados

- Java JDK-17
- org.OpenAPITools
- org.SpringDoc UI tools
- Springboot (2.7.11)
- Spring AMQP
- Apache Camel Orchestration (Spring framework)
- SpringBatch
- SpringCloud (with Eureka Netflix)
- SpringCloud (with SpringGateway)
- SpringSecurity (with JWT and OAuth2)
- Resilience4J
- Authentication Server (SpringSecurity framework)
- Authorization Server (SpringSecurity framework)
- MapStruct
- JUnit
- Lombok